systemd

From the **project web page**:

> *systemd* is a suite of basic building blocks for a Linux system. It provides a system and service manager that runs as PID 1 and starts the rest of the system. systemd provides aggressive parallelization capabilities, uses socket and **D-Bus** activation for starting services, offers on-demand starting of daemons, keeps track of processes using Linux **control groups**, maintains mount and automount points, and implements an elaborate transactional dependency-based service control logic. *systemd* supports SysV and LSB init scripts and works as a replacement for sysvinit. Other parts include a logging daemon, utilities to control basic system configuration like the hostname, date, locale, maintain a list of logged-in users and running containers and virtual machines, system accounts, runtime directories and settings, and daemons to manage simple network configuration, network time synchronization, log forwarding, and name resolution.

Note: For a detailed explanation of why Arch moved to *systemd*, see **this forum post**.

Related articles

systemd/User
systemd/Timers
systemd/Journal
systemd FAQ
init
Daemons
udev
Improving performance/Boot process
Allow users to shutdown

Contents [hide]

Basic systemctl usage

The main command used to introspect and control *systemd* is *systemctl*. Some of its uses are examining the system state and managing the system and services. See `systemctl(1)` for more details.

Tip:

- You can use all of the following *systemctl* commands with the `-H user@host` switch to control a *systemd* instance on a remote machine. This will use **SSH** to connect to the remote *systemd* instance.
- **Plasma** users can install `systemd-kcm`[AUR] as a graphical frontend for *systemctl*. After installing the module will be added under *System administration*.

Analyzing the system state

Show **system status** using:

```
$ systemctl status
```

List running units:

```
$ systemctl
```

or:

```
$ systemctl list-units
```

List failed units:

```
$ systemctl --failed
```

The available unit files can be seen in `/usr/lib/systemd/system/` and `/etc/systemd/system/` (the latter takes precedence). **List installed** unit files with:

```
$ systemctl list-unit-files
```

Show the **cgroup slice**, memory and parent for a PID:

```
$ systemctl status pid
```

Using units

Units can be, for example, services (*.service*), mount points (*.mount*), devices (*.device*) or sockets (*.socket*).

When using *systemctl*, you generally have to specify the complete name of the unit file, including its suffix, for example `sshd.socket`. There are however a few short forms when specifying the unit in the following *systemctl* commands:

- If you do not specify the suffix, systemctl will assume *.service*. For example, `netctl` and `netctl.service` are equivalent.
- Mount points will automatically be translated into the appropriate *.mount* unit. For example, specifying `/home` is equivalent to `home.mount`.
- Similar to mount points, devices are automatically translated into the appropriate *.device* unit, therefore specifying `/dev/sda2` is equivalent to `dev-sda2.device`.

See **`systemd.unit(5)`** for details.

Note: Some unit names contain an `@` sign (e.g. `name@string.service`): this means that they are **instances** of a *template* unit, whose actual file name does not contain the `string` part (e.g. `name@.service`). `string` is called the *instance identifier*, and is similar to an argument that is passed to the template unit when called with the *systemctl* command: in the unit file it will substitute the `%i` specifier. To be more accurate, *before* trying to instantiate the `name@.suffix` template unit, *systemd* will actually look for a unit with the exact `name@string.suffix` file name, although by convention such a "clash" happens rarely, i.e. most unit files containing an `@` sign are meant to be templates. Also, if a template unit is called without an instance identifier, it will just fail, since the `%i` specifier cannot be substituted.

Tip:

- Most of the following commands also work if multiple units are specified, see **`systemctl(1)`** for more information.
- The `--now` switch can be used in conjunction with `enable`, `disable`, and `mask` to respectively start, stop, or mask the unit *immediately* rather than after rebooting.
- A package may offer units for different purposes. If you just installed a package, `pacman -Qql package | grep -Fe .service -e .socket` can be used to check and find them.

Checking status

Manual pages associated with a unit (as supported by the unit) are shown with:

```
$ systemctl help unit
```

Status of a unit, including whether it is running or not is shown with:

```
$ systemctl status unit
```

Check whether a unit is enabled at or not:

```
$ systemctl is-enabled unit
```

Starting, restarting, reloading

Start a unit immediately:

```
# systemctl start unit
```

Stop a unit immediately:

```
# systemctl stop unit
```

Restart a unit:

```
# systemctl restart unit
```

Reload a unit and its configuration:

```
# systemctl reload unit
```

Reload systemd manager configuration, scanning for new or changed units:

```
# systemctl daemon-reload
```

Note: This does not ask the changed units to reload their own configurations. See **Reload** example above.

Enabling

Enable a unit to start automatically at boot:

```
# systemctl enable unit
```

Enable a unit to start automatically at boot and **start** it immediately:

```
# systemctl enable --now unit
```

Disable a unit to no longer start at boot:

```
# systemctl disable unit
```

Reenable (i.e. disable and enable anew) a unit; for example, in case its `[Install]` section has changed since last enabling it:

```
# systemctl reenable unit
```

Masking

Mask a unit to make it impossible to start (both manually and as a dependency, which makes masking dangerous):

```
# systemctl mask unit
```

Unmask a unit:

```
# systemctl unmask unit
```

Power management

polkit is necessary for power management as an unprivileged user. If you are in a local *systemd-logind* user session and no other session is active, the following commands will work without root privileges. If not (for example, because another user is logged into a tty), *systemd* will automatically ask you for the root password.

Shut down and reboot the system:

```
$ systemctl reboot
```

Shut down and power-off the system:

```
$ systemctl poweroff
```

Suspend the system:

```
$ systemctl suspend
```

Put the system into hibernation:

```
$ systemctl hibernate
```

Put the system into hybrid-sleep state (or suspend-to-both):

```
$ systemctl hybrid-sleep
```

Writing unit files

The syntax of *systemd*'s unit files (**`systemd.unit(5)`**) is inspired by **XDG Desktop Entry Specification .desktop files**, which are in turn inspired by **Microsoft Windows .ini files**. Unit files are loaded from multiple locations (to see the full list, run `systemctl show --property=UnitPath`), but the main ones are (listed from lowest to highest precedence):

- `/usr/lib/systemd/system/` : units provided by installed packages
- `/etc/systemd/system/` : units installed by the system administrator

Note:

- The load paths are completely different when running *systemd* in **user mode**.
- systemd unit names may only contain ASCII alphanumeric characters, underscores and periods. All other characters must be replaced by C-style "\x2d" escapes, or employ their predefined semantics ('@', '-'). See **`systemd.unit(5)`** and **`systemd-escape(1)`** for more information.

Look at the units installed by your packages for examples, as well as **`systemd.service(5)` § `EXAMPLES`**.

Tip: Comments prepended with `#` may be used in unit-files as well, but only in new lines. Do not use end-line comments after *systemd* parameters or the unit will fail to activate.

Handling dependencies

With *systemd*, dependencies can be resolved by designing the unit files correctly. The most typical case is that the unit *A* requires the unit *B* to be running before *A* is started. In that case add `Requires=B` and `After=B` to the `[Unit]` section of *A*. If the dependency is optional, add `Wants=B` and `After=B` instead. Note that `Wants=` and `Requires=` do

not imply `After=`, meaning that if `After=` is not specified, the two units will be started in parallel.

Dependencies are typically placed on services and not on **#Targets**. For example, `network.target` is pulled in by whatever service configures your network interfaces, therefore ordering your custom unit after it is sufficient since `network.target` is started anyway.

Service types

There are several different start-up types to consider when writing a custom service file. This is set with the `Type=` parameter in the `[Service]` section:

- `Type=simple` (default): *systemd* considers the service to be started up immediately. The process must not fork. Do not use this type if other services need to be ordered on this service, unless it is socket activated.
- `Type=forking`: *systemd* considers the service started up once the process forks and the parent has exited. For classic daemons use this type unless you know that it is not necessary. You should specify `PIDFile=` as well so *systemd* can keep track of the main process.
- `Type=oneshot`: this is useful for scripts that do a single job and then exit. You may want to set `RemainAfterExit=yes` as well so that *systemd* still considers the service as active after the process has exited.
- `Type=notify`: identical to `Type=simple`, but with the stipulation that the daemon will send a signal to *systemd* when it is ready. The reference implementation for this notification is provided by *libsystemd-daemon.so*.
- `Type=dbus`: the service is considered ready when the specified `BusName` appears on DBus's system bus.
- `Type=idle`: *systemd* will delay execution of the service binary until all jobs are dispatched. Other than that behavior is very similar to `Type=simple`.

See the **`systemd.service(5)` § `OPTIONS`** man page for a more detailed explanation of the `Type` values.

Editing provided units

This article or section needs language, wiki syntax or style improvements. See Help:Style for reference.

Reason: Should be renamed to more descriptive *Modifying provided units*. (Discuss in Talk:Edit#Deprecation)

To avoid conflicts with pacman, unit files provided by packages should not be directly edited. There are two safe ways to modify a unit without touching the original file: create a new unit file which **overrides the original unit** or create **drop-in snippets** which are applied on top of the original unit. For both methods, you must reload the unit afterwards to apply your changes. This can be done either by editing the unit with `systemctl edit` (which reloads the unit automatically) or by reloading all units with:

```
# systemctl daemon-reload
```

Tip:

- You can use *systemd-delta* to see which unit files have been overridden or extended and what exactly has been changed.
- Use `systemctl cat unit` to view the content of a unit file and all associated drop-in snippets.

Replacement unit files

To replace the unit file `/usr/lib/systemd/system/unit`, create the file `/etc/systemd/system/unit` and **reenable** the unit to update the symlinks.

Alternatively, run:

```
# systemctl edit --full unit
```

This opens `/etc/systemd/system/unit` in your editor (copying the installed version if it does not exist yet) and automatically reloads it when you finish editing.

Note: The replacement units will keep on being used even if Pacman updates the original units in the future. This method makes system maintenance more difficult and therefore the next approach is preferred.

Drop-in files

To create drop-in files for the unit file `/usr/lib/systemd/system/unit`, create the directory `/etc/systemd/system/unit.d/` and place *.conf* files there to override or add new options. *systemd* will parse and apply these files on top of the original unit.

The easiest way to do this is to run:

```
# systemctl edit unit
```

This opens the file `/etc/systemd/system/unit.d/override.conf` in your text editor (creating it if necessary) and automatically reloads the unit when you are done editing.

Note: Not all keys can be overridden with drop-in files. For example, for changing `Conflicts=` a replacement file **is necessary**.

Revert to vendor version

To revert any changes to a unit made using `systemctl edit` do:

```
# systemctl revert unit
```

Examples

For example, if you simply want to add an additional dependency to a unit, you may create the following file:

```
/etc/systemd/system/unit.d/customdependency.conf
```

```
[Unit]
Requires=new dependency
After=new dependency
```

As another example, in order to replace the `ExecStart` directive for a unit that is not of type `oneshot`, create the following file:

```
/etc/systemd/system/unit.d/customexec.conf
```

```
[Service]
ExecStart=
ExecStart=new command
```

Note how `ExecStart` must be cleared before being re-assigned [1]. The same holds for every item that can be specified multiple times, e.g. `OnCalendar` for timers.

One more example to automatically restart a service:

```
/etc/systemd/system/unit.d/restart.conf
```

```
[Service]
Restart=always
RestartSec=30
```

Targets

This article or section needs language, wiki syntax or style improvements. See Help:Style for reference.

Reason: Unclear description, copy-pasted content (explicitly mentions "Fedora"). (Discuss in "Targets"_more_clearly Talk:Systemd#Make section "Targets" more clearly)

systemd uses *targets* to group units together via dependencies and as standardized synchronization points. They serve a similar purpose as runlevels but act a little differently. Each *target* is named instead of numbered and is intended to serve a specific purpose with the possibility of having multiple ones active at the same time. Some *targets* are implemented by inheriting all of the services of another *target* and adding additional services to it. There are *systemd targets* that mimic the common SystemVinit runlevels so you can still switch *targets* using the familiar `telinit RUNLEVEL` command.

Get current targets

The following should be used under *systemd* instead of running `runlevel`:

```
$ systemctl list-units --type=target
```

Create custom target

The runlevels that held a defined meaning under sysvinit (i.e., 0, 1, 3, 5, and 6); have a 1:1 mapping with a specific *systemd target*. Unfortunately, there is no good way to do the same for the user-defined runlevels like 2 and 4. If you make use of those it is suggested that you make a new named *systemd target* as `/etc/systemd/system/your target` that takes one of the existing runlevels as a base (you can look at `/usr/lib/systemd/system/graphical.target` as an example), make a directory `/etc/systemd/system/your target.wants`, and then symlink the additional services from `/usr/lib/systemd/system/` that you wish to enable.

Mapping between SysV runlevels and systemd targets

SysV Runlevel	systemd Target	Notes
0	runlevel0.target, poweroff.target	Halt the system.
1, s, single	runlevel1.target, rescue.target	Single user mode.
2, 4	runlevel2.target, runlevel4.target, multi-user.target	User-defined/Site-specific runlevels. By default, identical to 3.
3	runlevel3.target, multi-user.target	Multi-user, non-graphical. Users can usually login via multiple consoles or via the network.
5	runlevel5.target, graphical.target	Multi-user, graphical. Usually has all the services of runlevel 3 plus a graphical login.
6	runlevel6.target, reboot.target	Reboot
emergency	emergency.target	Emergency shell

Change current target

In *systemd* targets are exposed via *target units*. You can change them like this:

```
# systemctl isolate graphical.target
```

This will only change the current target, and has no effect on the next boot. This is equivalent to commands such as `telinit 3` or `telinit 5` in Sysvinit.

Change default target to boot into

The standard target is `default.target`, which is a symlink to `graphical.target`. This roughly corresponds to the old runlevel 5.

To verify the current target with *systemctl*:

```
$ systemctl get-default
```

To change the default target to boot into, change the `default.target` symlink. With *systemctl*:

```
# systemctl set-default multi-user.target
```

```
Removed /etc/systemd/system/default.target.
Created symlink /etc/systemd/system/default.target -> /usr/lib/systemd/system/multi-user.target.
```

Alternatively, append one of the following kernel parameters to your bootloader:

- `systemd.unit=multi-user.target` (which roughly corresponds to the old runlevel 3),
- `systemd.unit=rescue.target` (which roughly corresponds to the old runlevel 1).

Default target order

Systemd chooses the `default.target` according to the following order:

1. Kernel parameter shown above
2. Symlink of `/etc/systemd/system/default.target`
3. Symlink of `/usr/lib/systemd/system/default.target`

systemd components

Some (not exhaustive) components of systemd are:

- **systemd-boot** — simple UEFI **boot manager**;
- **systemd-firstboot** — basic system setting initialization before first boot;
- **systemd-homed** — portable human-user **accounts**;
- **systemd-logind** — session management;
- **systemd-networkd** — **network configuration** management;
- **systemd-nspawn** — light-weight namespace container;
- **systemd-resolved** — network **name resolution**;
- **`systemd-sysusers(8)`** — creates system users and groups and adds users to groups at package installation or boot time;
- **systemd-timesyncd** — **system time** synchronization across the network;
- **systemd/Journal** — system logging;
- **systemd/Timers** — monotonic or realtime timers for controling `.service` files or events, reasonable alternative to **cron**.

systemd.mount - mounting

systemd is in charge of mounting the partitions and filesystems specified in `/etc/fstab`. The **`systemd-fstab-generator(8)`** translates all the entries in `/etc/fstab` into systemd units, this is performed at boot time and whenever the configuration of the system manager is reloaded.

systemd extends the usual **fstab** capabilities and offers additional mount options. These affect the dependencies of the mount unit, they can for example ensure that a mount is performed only once the network is up or only once another partition is mounted. The full list of specific *systemd* mount options, typically prefixed with `x-systemd.`, is detailed in **`systemd.mount(5)` § FSTAB**.

An example of these mount options in the context of *automounting*, which means mounting only when the resource is required rather than automatically at boot time, is provided in **fstab#Automount with systemd**.

GPT partition automounting

On a **GPT** partitioned disk **`systemd-gpt-auto-generator(8)`** will mount partitions following the **Discoverable Partitions Specification**, thus they can be omitted from `fstab`.

EFI system partition automounting requires that the boot loader sets the **LoaderDevicePartUUID** EFI variable. Only **systemd-boot** is known to do this.

For `/var` and `/var/tmp` automounting to work, the PARTUUID must match the SHA256 HMAC hash of the partition type UUID keyed by the machine ID. The required PARTUUID can be obtained using:

```
$ systemd-id128 -u --app-specific=partition-type-UUID machine-id
```

Replace *`partition-type-UUID`* with the appropriate partition type UUID value from the **Discoverable Partitions Specification**.

Note: **`systemd-id128(1)`** reads the machine ID from `/etc/machine-id`, this makes it impossible to know the needed PARTUUID before the system is installed.

The automounting of a partition can be disabled by changing the partition's **type GUID** or setting the partition attribute bit 63 "do not automount", see **gdisk#Prevent GPT partition automounting**.

systemd-sysvcompat

`systemd-sysvcompat` (required by **`base`**) primary role is to provide the traditional linux **init** binary. For systemd controlled systems, `init` is just a symbolic link to its `systemd` executable.

In addition, it also provides 4 convenience short cuts that **SysVinit** users might be used to. The convenience short cuts are **`halt(8)`**, **`poweroff(8)`**, **`reboot(8)`** and **`shutdown(8)`**. Each one of those 4 commands is a symbolic link to `systemctl`, and governed by systemd behavior. Therefore, the discussion at **#Power management** applies.

Systemd based systems can give up those System V compatibility methods by using the `init=` **boot parameter** (see, for example, **[solved] /bin/init is in systemd-sysvcompat ?**) and systemd native `systemctl` command arguments.

systemd-tmpfiles - temporary files

"*systemd-tmpfiles* creates, deletes and cleans up volatile and temporary files and directories." It reads configuration files in `/etc/tmpfiles.d/` and `/usr/lib/tmpfiles.d/` to discover which actions to perform. Configuration files in the former directory take precedence over those in the latter directory.

Configuration files are usually provided together with service files, and they are named in the style of `/usr/lib/tmpfiles.d/program.conf`. For example, the **Samba** daemon expects the directory `/run/samba` to exist and to have the correct permissions. Therefore, the **`samba`** package ships with this configuration:

```
/usr/lib/tmpfiles.d/samba.conf
```

```
D /run/samba 0755 root root
```

Configuration files may also be used to write values into certain files on boot. For example, if you used `/etc/rc.local` to disable wakeup from USB devices with `echo USBE > /proc/acpi/wakeup`, you may use the following tmpfile instead:

```
/etc/tmpfiles.d/disable-usb-wake.conf
```

```
#    Path                 Mode UID  GID  Age Argument
w    /proc/acpi/wakeup    -    -    -    -   USBE
```

See the **`systemd-tmpfiles(8)`** and **`tmpfiles.d(5)`** man pages for details.

Note: This method may not work to set options in `/sys` since the *systemd-tmpfiles-setup* service may run before the appropriate device modules is loaded. In this case you could check whether the module has a parameter for the option you want to set with `modinfo module` and set this option with a **config file in /etc/modprobe.d**. Otherwise you will have to write a **udev rule** to set the appropriate attribute as soon as the device appears.

Tips and tricks

GUI configuration tools

- **systemadm** — Graphical browser for systemd units. It can show the list of units, possibly filtered by type.
  https://cgit.freedesktop.org/systemd/systemd-ui/ || **`systemd-ui`**
- **SystemdGenie** — systemd management utility based on KDE technologies.
  https://invent.kde.org/system/systemdgenie || **`systemdgenie`**

Running services after the network is up

To delay a service until after the network is up, include the following dependencies in the *.service* file:

/etc/systemd/system/*foo*.service

```
[Unit]
...
Wants=network-online.target
After=network-online.target
...
```

The network wait service of the particular application that manages the network, must also be enabled so that `network-online.target` properly reflects the network status.

- If using **NetworkManager**, `NetworkManager-wait-online.service` is enabled together with `NetworkManager.service`. Check if this is the case with `systemctl is-enabled NetworkManager-wait-online.service`. If it is not enabled, then **reenable** `NetworkManager.service`.
- In the case of **netctl**, **enable** the `netctl-wait-online.service`.
- If using **systemd-networkd**, `systemd-networkd-wait-online.service` is enabled together with `systemd-networkd.service`. Check if this is the case with `systemctl is-enabled systemd-networkd-wait-online.service`. If it is not enabled, then **reenable** `systemd-networkd.service`.

For more detailed explanations see **Running services after the network is up** in the systemd wiki.

If a service needs to perform DNS queries, it should additionally be ordered after `nss-lookup.target`:

/etc/systemd/system/*foo*.service

```
[Unit]
...
Wants=network-online.target
After=network-online.target nss-lookup.target
...
```

See **`systemd.special(7)` § Special Passive System Units**.

For `nss-lookup.target` to have any effect it needs a service that pulls it in via `Wants=nss-lookup.target` and orders itself before it with `Before=nss-lookup.target`. Typically this is done by local **DNS resolvers**.

Check which active service, if any, is pulling in `nss-lookup.target` with:

```
$ systemctl list-dependencies --reverse nss-lookup.target
```

Enable installed units by default

This article or section needs expansion.

Reason: How does it work with instantiated units? (Discuss in **Talk:Systemd#**)

Arch Linux ships with `/usr/lib/systemd/system-preset/99-default.preset` containing `disable *`. This causes *systemctl preset* to disable all units by default, such that when a new package is installed, the user must manually enable the unit.

If this behavior is not desired, simply create a symlink from `/etc/systemd/system-preset/99-default.preset` to `/dev/null` in order to override the configuration file. This will cause *systemctl preset* to enable all units that get installed—regardless of unit type—unless specified in another file in one *systemctl preset*'s configuration directories. User units are not affected. See **`systemd.preset(5)`** for more information.

Note: Enabling all units by default may cause problems with packages that contain two or more mutually exclusive units. *systemctl preset* is designed to be used by distributions and spins or system administrators. In the case where two conflicting units would be enabled, you should explicitly specify which one is to be disabled in a preset configuration file as specified in the **`systemd.preset(5)`** man page.

Sandboxing application environments

A unit file can be created as a sandbox to isolate applications and their processes within a hardened virtual environment. systemd leverages **namespaces**, a list of allowed/denied **capabilities**, and **control groups** to container processes through an extensive execution environment configuration—**`systemd.exec(5)`**.

The enhancement of an existing systemd unit file with application sandboxing typically requires trial-and-error tests accompanied by the generous use of **`strace`**, **stderr** and **`journalctl(1)`** error logging and output facilities. You may want to first search upstream documentation for already done tests to base trials on. To get a starting point for possible hardening options, run

```
$ systemd-analyze security unit
```

Some examples of how sandboxing with systemd can be deployed:

- `CapabilityBoundingSet` defines a list of **`capabilities(7)`** that are allowed or denied for a unit. See **`systemd.exec(5)` § CAPABILITIES**.
  - The `CAP_SYS_ADM` capability, for example, which should be one of the **goals of a secure sandbox**: `CapabilityBoundingSet=~ CAP_SYS_ADM`

Notifying about failed services

In order to notify about service failures, a `OnFailure=` directive needs to be added to the according service file, for example by using a **drop-in configuration file**. Adding this directive to every service unit can be achieved with a top-level drop-in configuration file. For details about top-level drop-ins, see **`systemd.unit(5)`**.

Create a top-level drop-in for services:

/etc/systemd/system/service.d/toplevel-override.conf

```
[Unit]
OnFailure=failure-notification@%n
```

This adds `OnFailure=failure-notification@%n` to every service file. If *some_service_unit* fails, `failure-notification@`*some_service_unit* will be started to handle the notification delivery (or whatever task it is configured to perform).

Create the `failure-notification@` template unit:

/etc/systemd/system/failure-notification@.service

```
[Unit]
Description=Send a notification about a failed systemd unit
After=network.target

[Service]
Type=simple
ExecStart=/path/to/failure-notification.sh %i
```

You can create the `failure-notification.sh` script and define what to do or how to notify (mail, gotify, xmpp, etc.). The `%i` will be the name of the failed service unit and will be passed as argument to the script.

In order to prevent a regression for instances of `failure-notification@.service`, create an empty drop-in configuration file with the same name as the top-level drop-in (the empty service-level drop-in configuration file takes precedence over the top-level drop-in and overrides the latter one):

```
# mkdir -p /etc/systemd/system/failure-notification@.service.d
# touch /etc/systemd/system/failure-notification@.service.d/toplevel-override.conf
```

Troubleshooting

Investigating failed services

To find the *systemd* services which failed to start:

```
$ systemctl --state=failed
```

To find out why they failed, examine their log output. See **systemd/Journal#Filtering output** for details.

Diagnosing boot problems

systemd has several options for diagnosing problems with the boot process. See **boot debugging** for more general instructions and options to capture boot messages before *systemd* takes over the **boot process**. Also see the **systemd debugging documentation**.

Diagnosing a service

If some *systemd* service misbehaves or you want to get more information about what is happening, set the `SYSTEMD_LOG_LEVEL` **environment variable** to `debug`. For example, to run the *systemd-networkd* daemon in debug mode:

Add a **drop-in file** for the service adding the two lines:

```
[Service]
Environment=SYSTEMD_LOG_LEVEL=debug
```

Or equivalently, set the environment variable manually:

```
# SYSTEMD_LOG_LEVEL=debug /lib/systemd/systemd-networkd
```

then **restart** *systemd-networkd* and watch the journal for the service with the `-f`/`--follow` option.

Shutdown/reboot takes terribly long

If the shutdown process takes a very long time (or seems to freeze) most likely a service not exiting is to blame. *systemd* waits some time for each service to exit before trying to kill it. To find out if you are affected, see **this article**.

Short lived processes do not seem to log any output

If `journalctl -u foounit` does not show any output for a short lived service, look at the PID instead. For example, if `systemd-modules-load.service` fails, and `systemctl status systemd-modules-load` shows that it ran as PID 123, then you might be able to see output in the journal for that PID, i.e. `journalctl -b _PID=123`. Metadata fields for the journal such as `_SYSTEMD_UNIT` and `_COMM` are collected asynchronously and rely on the `/proc` directory for the process existing. Fixing this requires fixing the kernel to provide this data via a socket connection, similar to `SCM_CREDENTIALS`. In short, it is a **bug**. Keep in mind that immediately failed services might not print anything to the journal as per design of systemd.

Boot time increasing over time

The factual accuracy of this article or section is disputed.

Reason: NetworkManager issues are not systemd's fault, the alleged reports are missing. Slow `systemctl status` or `journalctl` do not affect boot time. (Discuss in **Talk:Systemd#**)

After using `systemd-analyze` a number of users have noticed that their boot time has increased significantly in comparison with what it used to be. After using `systemd-analyze blame` **NetworkManager** is being reported as taking an unusually large amount of time to start.

The problem for some users has been due to `/var/log/journal` becoming too large. This may have other impacts on performance, such as for `systemctl status` or `journalctl`. As such the solution is to remove every file within the folder (ideally making a backup of it somewhere, at least temporarily) and then setting a journal file size limit as described in **Systemd/Journal#Journal size limit**.

systemd-tmpfiles-setup.service fails to start at boot

Starting with systemd 219, `/usr/lib/tmpfiles.d/systemd.conf` specifies ACL attributes for directories under `/var/log/journal` and, therefore, requires ACL support to be enabled for the filesystem the journal resides on.

See **Access Control Lists#Enable ACL** for instructions on how to enable ACL on the filesystem that houses `/var/log/journal`.

Disable emergency mode on remote machine

You may want to disable emergency mode on a remote machine, for example, a virtual machine hosted at Azure or Google Cloud. It is because if emergency mode is triggered, the machine will be blocked from connecting to network.

```
# systemctl mask emergency.service
# systemctl mask emergency.target
```

See also

- **Wikipedia:systemd**
- **Official web site**
  - **systemd optimizations**
  - **systemd FAQ**
  - **systemd Tips and tricks**
- **Manual pages**
- Other distributions
  - **Gentoo:Systemd**
  - **Fedora:Systemd**
  - **Fedora:How to debug Systemd problems**
  - **Fedora:SysVinit to Systemd Cheatsheet**
  - **Debian:systemd**
- **Lennart's blog story**, **update 1**, **update 2**, **update 3**, **summary**
- **Debug Systemd Services**
- **systemd for Administrators (PDF)**
- **How To Use Systemctl to Manage Systemd Services and Units**
- **Session management with systemd-logind**
- **Emacs Syntax highlighting for Systemd files**
- **Two** **part** introductory article in *The H Open* magazine.

Category: Init